
Docker

Prof. Lázaro

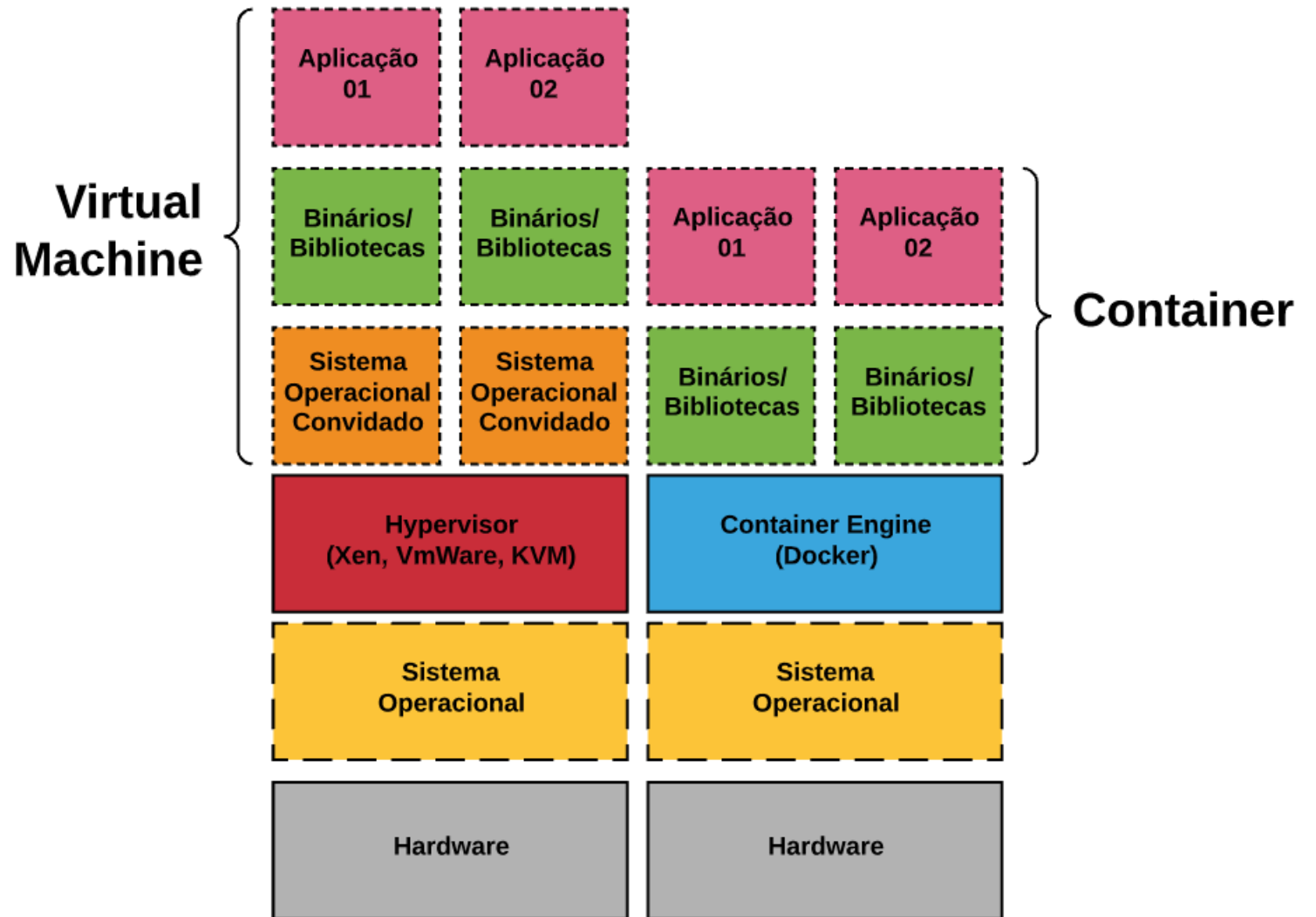
Fundamentos

Docker é uma plataforma de código aberto que permite aos desenvolvedores empacotar, distribuir e executar aplicações em containers.

A containerização revolucionou o desenvolvimento de software ao permitir que aplicações sejam executadas de forma isolada e portátil em qualquer ambiente que suporte Docker.

VM x Container

- Um container inclui apenas a aplicação e suas dependências, resultando em inicialização mais rápida e menor consumo de recursos.



Arquitetura

- Segue um modelo cliente-servidor composto por três componentes principais:
 - Docker Daemon (dockerd): O processo que executa em segundo plano no host, gerenciando imagens, containers, redes e volumes. Escuta requisições da API Docker e se comunica com outros daemons quando necessário.
 - Docker Client: Interface de linha de comando (CLI) que permite aos usuários interagir com o daemon através de comandos como `docker build`, `docker run`, e `docker pull`.
 - Docker Registry: Sistema de armazenamento para imagens Docker, sendo o Docker Hub o registro público mais popular.

Images e Containers

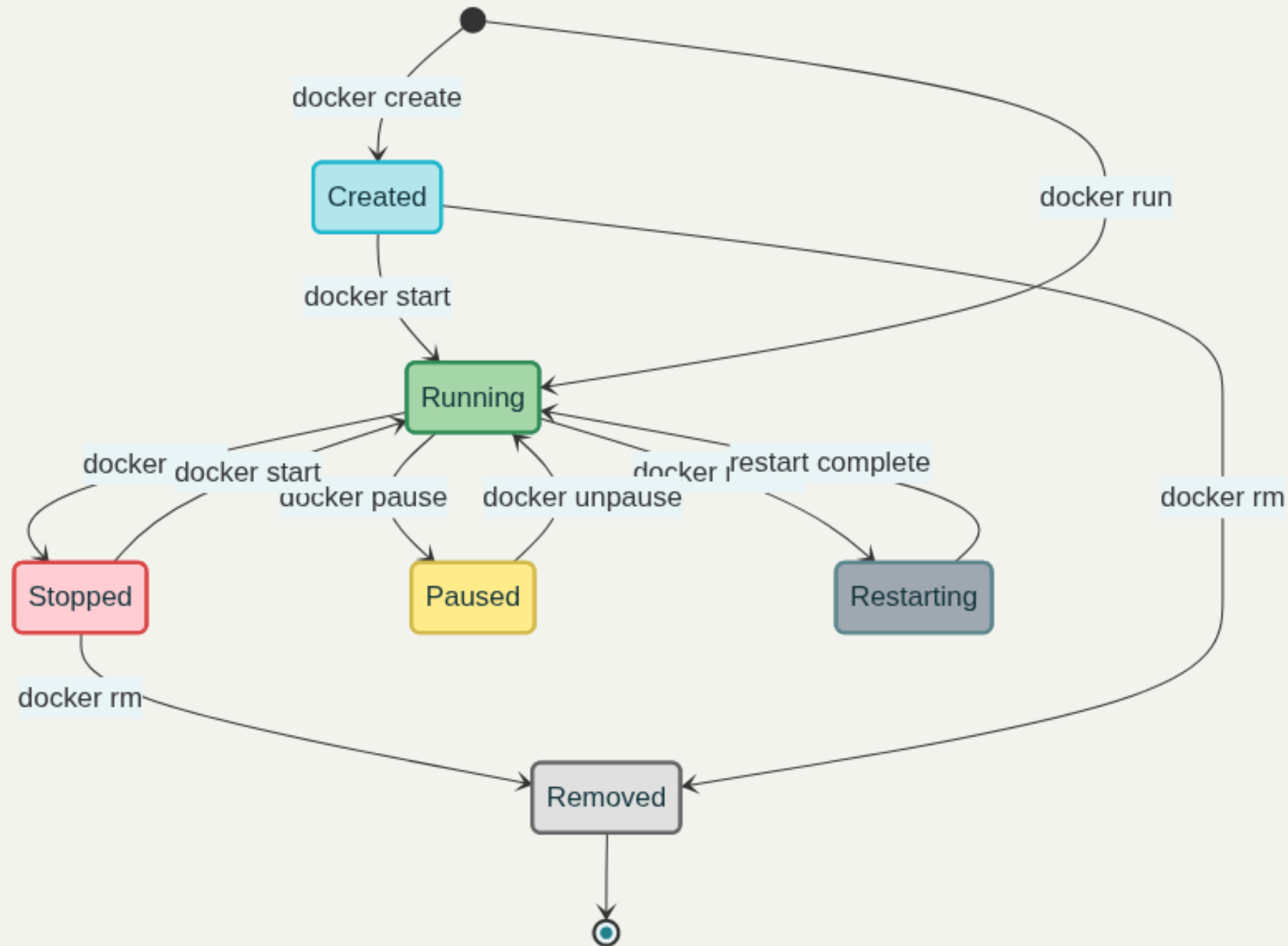
Docker Images:

- São templates imutáveis que contêm todos os arquivos, binários, bibliotecas e configurações necessárias para executar um container.
- As imagens são compostas por camadas (layers), onde cada camada representa um conjunto de mudanças no sistema de arquivos.

Docker Containers:

- São instâncias em execução de uma imagem.
- São ambientes isolados e efêmeros que podem ser iniciados, pausados e destruídos conforme necessário.
- Cada container possui sua própria camada de escrita sobre as camadas imutáveis da imagem base.

Ciclo de Vida dos Containers Docker



Docker Hub e Registries

O Docker Hub é o registro público padrão que hospeda mais de 100.000 imagens criadas pela comunidade.

Oferece diferentes tipos de conteúdo confiável:

- Docker Official Images: Imagens curadas que servem como ponto de partida seguro
- Docker Verified Publishers: Imagens de alta qualidade de publishers comerciais verificados
- Docker-Sponsored Open Source: Imagens mantidas por projetos open source patrocinados

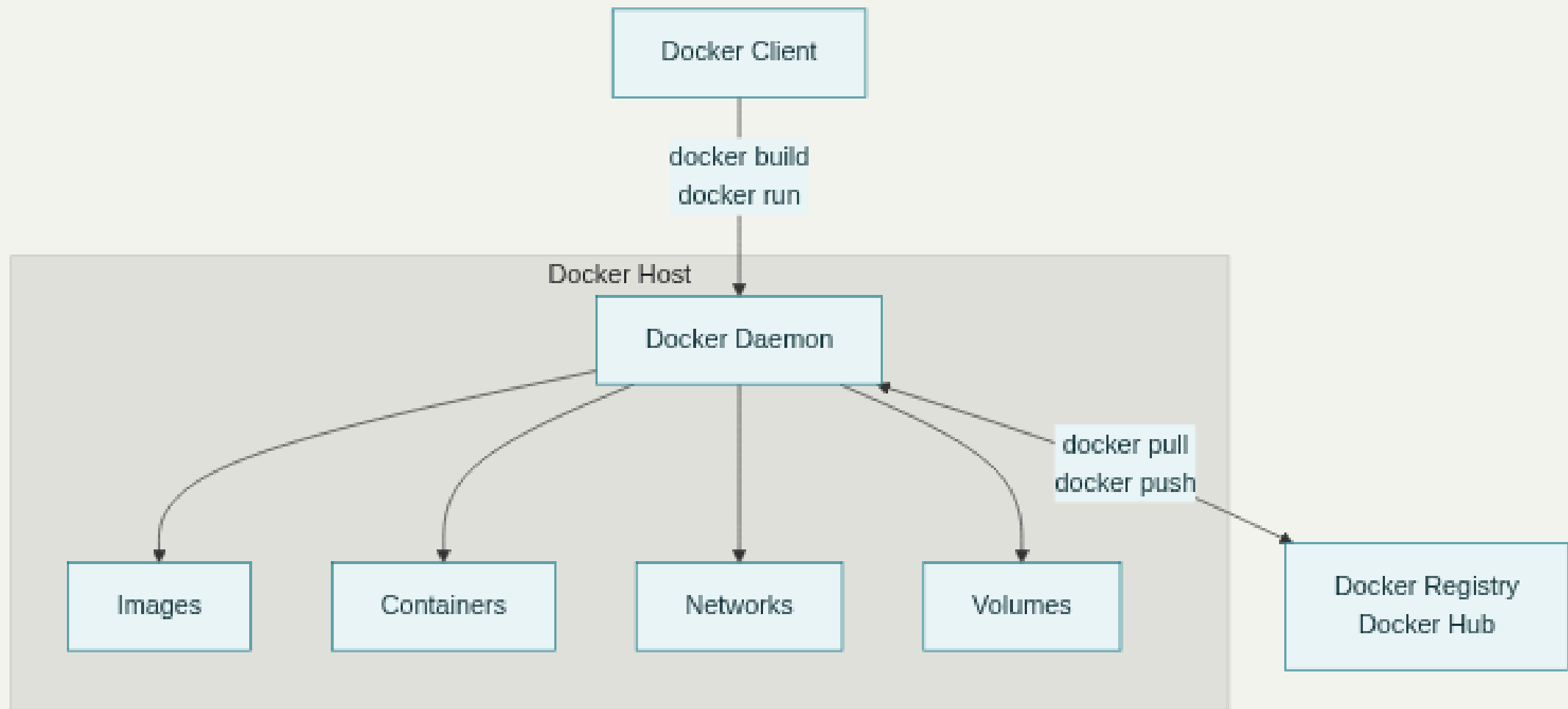
Para uso empresarial, organizações frequentemente implementam registries privados para controle de segurança e compliance.

Docker Network

- Bridge Network: Rede padrão que cria uma ponte software entre o host e containers, permitindo comunicação isolada.
- Host Network: Containers compartilham diretamente a pilha de rede do host, oferecendo melhor performance mas menor isolamento.
- Overlay Network: Permite comunicação segura entre containers executando em diferentes hosts Docker, essencial para clusters.
- None Network: Desabilita completamente o networking, usado para containers que não precisam de conectividade.

Gerenciamento de Dados com Volumes

- Os volumes Docker são o mecanismo recomendado para persistir dados gerados por containers.
- Diferente dos bind mounts, volumes são completamente gerenciados pelo Docker e oferecem vantagens significativas:
 - Facilidade de backup e migração
 - Gerenciamento via CLI ou API Docker
 - Funcionamento em Linux e Windows
 - Compartilhamento seguro entre múltiplos containers
 - Drivers de volume para armazenamento remoto



Arquitetura do Docker – Componentes principais e suas interações

Ambiente de desenvolvimento local

services:

web:

build:

context: .

dockerfile: Dockerfile.dev

volumes:

- ./code

- node_modules:/code/node_modules

ports:

- "3000:3000"

environment:

- NODE_ENV=development

depends_on:

- db

db:

image: postgres:15

environment:

POSTGRES_DB: myapp

POSTGRES_USER: developer

POSTGRES_PASSWORD: devpass

volumes:

- postgres_dev:/var/lib/postgresql/data

ports:

- "5432:5432"

volumes:

node_modules:

postgres_dev:

Gerenciamento de imagens

Build de imagem com tag

```
docker build -t myapp:1.0 .
```

Pull de imagem do registry

```
docker pull nginx:alpine
```

Push para registry

```
docker push myregistry.com/myapp:1.0
```

Listar imagens

```
docker images
```

Remover imagens não utilizadas

```
docker image prune -a
```

Gerenciamento de containers

Executar container em background

```
docker run -d --name webapp -p 80:80 nginx
```

Listar containers (ativos e inativos)

```
docker ps -a
```

Acessar shell do container

```
docker exec -it webapp bash
```

Ver logs do container

```
docker logs -f webapp
```

Parar e remover container

```
docker stop webapp && docker rm webapp
```

Gerenciamento de Volumes e Redes

Criar volume nomeado

```
docker volume create mydata
```

Criar rede customizada

```
docker network create --driver bridge mynetwork
```

Conectar container à rede

```
docker network connect mynetwork webapp
```

Multi-Stage Builds

- Multi-stage builds permitem usar múltiplas instruções FROM em um único Dockerfile, separando ambientes de build e runtime. Esta técnica resulta em imagens menores e mais seguras:
- Os benefícios incluem:
 - Redução significativa do tamanho das imagens finais
 - Eliminação de ferramentas de desenvolvimento da imagem de produção
 - Melhor segurança ao reduzir a superfície de ataque
 - Builds paralelos para maior eficiência

```
# Estágio de build
FROM node:16-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
```

```
# Estágio de runtime
FROM node:16-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Práticas de Segurança

- **Imagens Base Seguras:** Use sempre imagens oficiais e verificadas. Evite imagens não testadas ou de fontes não confiáveis.
- **Usuários Não-Root:** Execute containers com usuários não privilegiados sempre que possível:

`RUN adduser --disabled-password --gecos "" appuser`

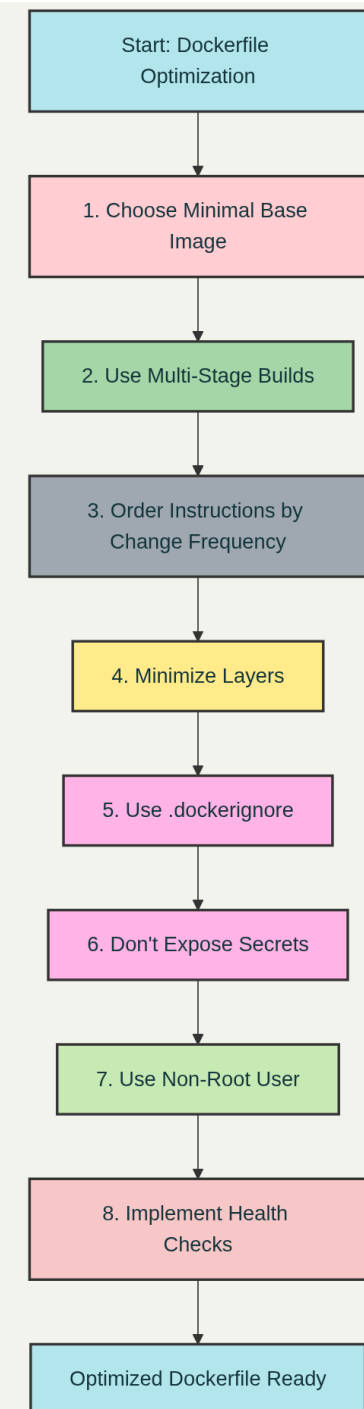
`USER appuser`

- **Limitação de Recursos:** Implemente limites de CPU, memória e outros recursos para prevenir ataques de negação de serviço:

`docker run -m 512m --cpus="0.5" myapp`

- **Filesystem Somente-Leitura:** Use containers com sistema de arquivos read-only quando apropriado:

`docker run --read-only --tmpfs /tmp myapp`



Otimização de Dockerfile

Ordenação Adequada de Comandos: Coloque comandos que mudam frequentemente no final para aproveitar o cache de layers.

Minimização de Layers: Combine comandos RUN relacionados para reduzir o número de layers.

Uso do .dockerignore: Exclua arquivos desnecessários do contexto de build.

Imagens Base Mínimas: Prefira imagens Alpine ou distroless para reduzir tamanho e superfície de ataque.

Mantenha imagens base atualizadas regularmente

Orquestração e Deploy

- Docker Swarm é a solução nativa de orquestração que transforma múltiplos Docker Engines em um cluster virtual.
- Características principais:
 - Inicialização Simples: `docker swarm init --advertise-addr <IP>`
 - Descoberta de Serviços: Automática entre containers no cluster
 - Load Balancing: Distribuição automática de carga
 - Scaling: Ajuste dinâmico do número de réplicas
 - Rolling Updates: Atualizações sem downtime

Workflows de Produção

- Pipeline CI/CD com Docker:
 - Build: Construção automatizada da imagem
 - Test: Execução de testes em container isolado
 - Scan: Análise de vulnerabilidades
 - Push: Upload para registry privado
 - Deploy: Deployment usando orchestrator

Monitoramento de Produção:

Estatísticas em tempo real

`docker stats`

Informações detalhadas do sistema

`docker system df`

Limpeza de recursos não utilizados

`docker system prune -a --volumes`

Debugging e Troubleshooting

Análise de Performance:

Inspecionar container

```
docker inspect webapp
```

Processos em execução no container

```
docker top webapp
```

Uso de recursos

```
docker stats webapp --no-stream
```

Análise de Logs:

Logs com timestamp

```
docker logs -t webapp
```

Seguir logs em tempo real

```
docker logs -f --since=1h webapp
```

Limitar número de linhas

```
docker logs --tail 100 webapp
```

Deploy de Produção Exemplo de deployment com health checks e resource limits:

```
version: '3.8'

services:

  app:
    image: myregistry.com/myapp:latest
    deploy:
      replicas: 3
    resources:
      limits:
        cpus: '0.5'
        memory: 512M
      reservations:
        cpus: '0.25'
        memory: 256M
    restart_policy:
      condition: on-failure
      delay: 5s
      max_attempts: 3

    ports:
      - "80:3000"

    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:3000/health"]
      interval: 30s
      timeout: 10s
      retries: 3

    networks:
      - webnet

networks:
  webnet:
    driver: overlay
```

Microserviços com Service Discovery

Implementação de arquitetura de microserviços:

version: '3.8'

services:

api-gateway:

image: nginx:alpine

ports:

- "80:80"

configs:

- nginx.conf

depends_on:

- user-service

- order-service

user-service:

image: myapp/user-service:latest

environment:

-

DATABASE_URL=postgresql://postgres:password@
user-db:5432/users

depends_on:

- user-db

order-service:

image: myapp/order-service:latest

environment:

-

DATABASE_URL=postgresql://postgres:password@
order-db:5432/orders

depends_on:

- order-db

user-db:

image: postgres:15

environment:

POSTGRES_DB: users

POSTGRES_PASSWORD: password

volumes:

- user_data:/var/lib/postgresql/data

order-db:

image: postgres:15

environment:

POSTGRES_DB: orders

POSTGRES_PASSWORD: password

volumes:

- order_data:/var/lib/postgresql/data

configs:

nginx.conf:

external: true

volumes:

user_data:

order_data:

Kubernetes

- Kubernetes utiliza Docker como runtime de containers através da Container Runtime Interface (CRI).
- A integração permite:
 - Orquestração avançada de containers Docker
 - Auto-scaling horizontal e vertical
 - Service mesh e políticas de rede granulares
 - Gerenciamento declarativo via YAML